Manual for solar cell optical simulation software: GENPRO4

Rudi Santbergen (r.santbergen@tudelft.nl)
Photovoltaic Materials and Devices
Delft University of Technology

Version: January 2022

1. Introduction

GENPRO4 is an optical model for solar cells that combines wave and ray-optics in a computationally efficient way. The model has been validated for a wide range of wafer based and thin-film solar cells. GENPRO4 represents the solar cell as a multilayer structure and calculates the fraction of incident light absorbed in each layer, taking into account interference and scattering of light at the interfaces. From this, the implied photocurrent and generation profile is calculated. Note that GENPRO4 is a purely *optical* model and does not consider the *electrical* characteristics of the solar cell. Also note that GENPRO4 is programmed in Matlab and requires Matlab to run.

Below some examples are provided to illustrate how to compose the input file, run the simulation and extract the output data. Details about the working principles and equations used in GENPRO4 can be found in ref. 1.

2. Basic simulations

Example 1: Polished c-Si wafer

The goal of this example is to simulate the reflectance *R*, transmittance *T* and absorptance *A* of double side polished c-Si wafer. Fig. 1a shows the input file 'example1.m'. The first line clears any input structures that may have been left in Matlab's workspace from previous simulations. The input 'Lay' is a structure with layer information, specifying the medium 'med' and thickness 'thi' of each layer in µm. This structure consists of 3 layers (air/c-Si/air), of which the first and final are infinitely thick. The name of the materials 'air' and 'c-Si' corresponds with the files 'air.nk' and 'c-Si.nk' in the nk-folder. Finally, the command 'GENPRO4(Lay);' runs the simulation with the input 'Lay'.

This file can be run by Matlab as a script by pressing the F5 key or the 'run' button in Matlab. This simulation should finish in a few seconds[1] and the output is shown in Fig. 2b. The results show *R* (white), *T* (light grey) and *A* (grey) as a function of wavelength. As expected, shorter wavelengths ($\lambda < 900$ nm) are either absorbed or reflected but not transmitted. The largest wavelengths ($\lambda \approx 1200$ nm) are reflected and transmitted but not absorbed. It can be checked that energy is conserved because for every wavelength $R + T + \Sigma A_i = 1$. GENPRO4 automatically integrates *R*, *T* and A_i over the AM1.5 spectrum to calculate the corresponding implied photocurrents (in mA/cm^2). These are shown in the legend of Fig. 1b.

(a)

```
clear Lay                   %clear variable 'Lay'

%===LAYERS (material & thickness [um])===
Lay(1).med = 'air';         Lay(1).thi = inf;
Lay(2).med = 'c-Si';        Lay(2).thi = 200;
Lay(3).med = 'air';         Lay(3).thi = inf;
%===
%Input data taken from 'air.nk' and 'c-Si.nk'

GENPRO4(Lay);               %run simulation
```

(b)

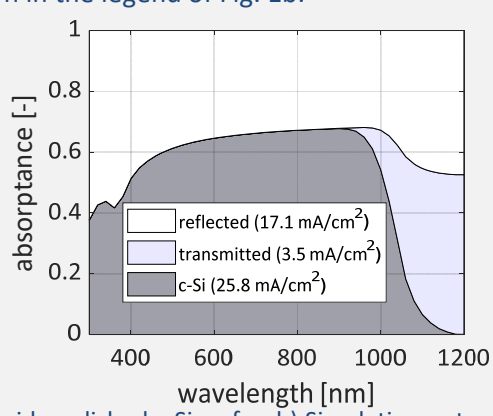


Fig. 1: a) Input file 'example1.m' for simulating a double side polished c-Si wafer. b) Simulation output: an area plot of *R*, *T* and A_i versus wavelength.

[1] *GENPRO4 can use Matlab's Parallel Computing Toolbox, if available, to distribute the calculation over the available CPU cores. When run for the first time a parallel pool is created, which could take a minute.*

Example 2: Flat silicon heterojunction cell

The goal of this example is to simulate a silicon heterojunction (SHJ) cell with optically flat interfaces. The SHJ cell is based on the polished c-Si wafer of example 1, but with several thin layers deposited on both sides. It has an 80 nm thin ITO coating that works as an anti-reflective coating due to interference. **To include the interference effect in the simulation, the ITO and other thin layers are *not* input as a 'layer' but as a 'coating'.** The coating is considered part of the interface and therefore the interface structure 'Int.coat' is used. On both front and rear there is a multilayer coating. At the front: ITO (80 nm)/a-Si(p) (5 nm)/a-Si(i) (5 nm) and at the rear a-Si(i) (5 nm)/a-Si(p) (5 nm)/silver (300 nm).

The result of the simulation is shown in Fig. 3b. As expected, the anti-reflection effect of the ITO reduces *R* (white area) considerably and as a result the absorption in c-Si increased (grey area). As indicated in the legend, the implied photocurrent of the c-Si wafer increased to 33.5 mA/cm^2 (this was 25.8 mA/cm^2). The other colours indicate that there is a small amount of parasitic absorption in the other layers as well. Because the silver coating at the rear is opaque, no light is transmitted.

(a)

```
clear Lay Int              %clear variables 'Lay' and 'Int'

%===LAYERS===
Lay(1).med = 'air';                Lay(1).thi = inf;
Lay(2).med = 'c-Si';               Lay(2).thi = 200;
Lay(3).med = 'air';                Lay(3).thi = inf;

%===INTERFACES===          coatings are part of the interface
%interface 1: between layer 1 and 2 (air/c-Si)
Int(1).coat(1).med = 'ITO';        Int(1).coat(1).thi = 0.080;
Int(1).coat(2).med = 'a-Si(p)';    Int(1).coat(2).thi = 0.005;
Int(1).coat(3).med = 'a-Si(i)';    Int(1).coat(3).thi = 0.005;
%interface 2: between layer 2 and 3 (c-Si/air)
Int(2).coat(1).med = 'a-Si(i)';    Int(2).coat(1).thi = 0.005;
Int(2).coat(2).med = 'a-Si(n)';    Int(2).coat(2).thi = 0.005;
Int(2).coat(3).med = 'Ag';         Int(2).coat(3).thi = 0.300;
%Note: interference occurs in 'coatings', not in 'layers'.
%===

GENPRO4(Lay,Int);          %both 'Lay' and 'Int' are input
```

(b)

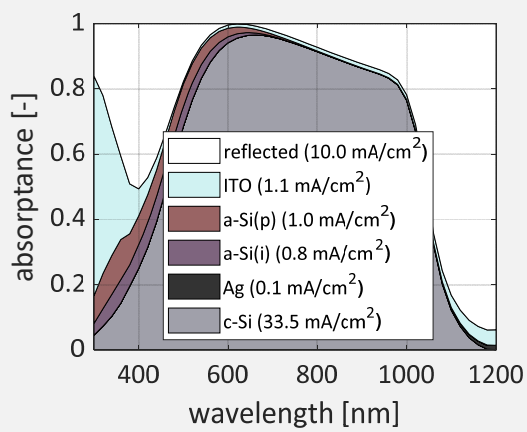


Fig. 2: a) Input file 'example2.m' for simulating a flat SHJ cell. b) Simulation output: an area plot of *R*, *T* and A_i versus wavelength.

Example 3: Pyramid textured silicon heterojunction cell

The goal of this example is to simulate the SHJ cell of example 2, but with a pyramid texture on both sides. The input for this simulation is shown in Fig. 3a. As in example 2, there are three 'layers' (air/c-Si/air) and the 'coatings' are part of the interfaces. The pyramid texture consists of approximately 5 µm wide pyramids with a steepness of 54.7°. These large features should be modelled with the **ray-optics model** therefore the model **'ray'** is enabled for both interfaces 1 and 2. The surface morphology shown in Fig. 3b is available in the form of a height matrix stored in the variable 'pyramids_20um', which is loaded from a file and assigned to the field 'Z'. Note that '20um' refers to the lateral size of the entire height map. This value has to be specified in the field 'xy'. Note that at the second interface, a minus-sign is added to assure that on the bottom interface the pyramids are pointing downward. Also note that all coatings, 'ITO', 'a-Si(p)' etc., are always conformal. This means that coatings perfectly follow the texture morphology and have a uniform thickness. Only layers (not coatings) can be non-conformal, i.e. have different texture morphology at their front and rear side.

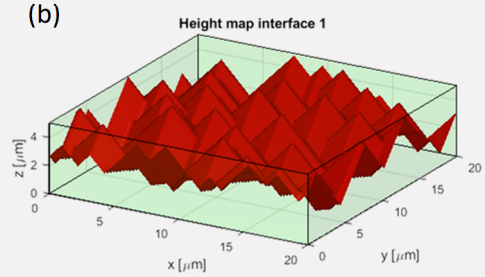
The simulation result shown in Fig. 5c reveals that the pyramid texture has a strong anti-reflective effect and even more light is absorbed in c-Si (grey area). As a result, the implied photocurrent of this SHJ cell increased from 33.5 mA/cm^2 to 38.9 mA/cm^2. The current densities of the other layers give an indication of the parasitic absorption losses.

The settings struct 'S' can be used to override the default simulation settings found in the file 'gp4_settings.m'. In this case the default 20 nm wavelength step is set to 50 nm. Note that 'S' has to be given as the third input parameter 'GENRPO4(Lay,Int,S)' to take effect. In addition, '[Lay,Int,out] =' was added to collect the output in the variables 'Lay', 'Int' and 'out' in the Matlab workspace. This data is then available, e.g. to be copied or exported to a file.

(a)

```
clear Lay Int                          %clear variables

load('AFM.mat','pyramids_20um')  %load 20x20um height map
%height map of typical pyramid texture (~5um pyramid size)

%===LAYERS===
Lay(1).med = 'air';                    Lay(1).thi = inf;
Lay(2).med = 'c-Si';                   Lay(2).thi = 200;
Lay(3).med = 'air';                    Lay(3).thi = inf;

%===INTERFACES===
Int(1).model = 'ray';                  %use RAY-optics model
Int(1).Z = pyramids_20um;              Int(1).xy = [20,20];

Int(1).coat(1).med = 'ITO';            Int(1).coat(1).thi = 0.080;
Int(1).coat(2).med = 'a-Si(p)';        Int(1).coat(2).thi = 0.005;
Int(1).coat(3).med = 'a-Si(i)';        Int(1).coat(3).thi = 0.005;
%---

Int(2).model = 'ray';                  %use RAY-optics model
Int(2).Z = -pyramids_20um;             Int(2).xy = [20,20];
%minus was added to get upside-down height map for rear

Int(2).coat(1).med = 'a-Si(i)';        Int(2).coat(1).thi = 0.005;
Int(2).coat(2).med = 'a-Si(n)';        Int(2).coat(2).thi = 0.005;
Int(2).coat(3).med = 'Ag';             Int(2).coat(3).thi = 0.300;
%===

%---override default settings (see 'gp4_settings.m')---
S.wav = 0.300:0.050:1.200;             %wavelengths [um]
S.SM = 1;                              %output SM (see example4)

[Lay,Int,out] = GENPRO4(Lay,Int,S);  %'S' is 3rd input
%simulation output is stored in 'Lay', 'Int' and 'out'
```

(b)



(c)

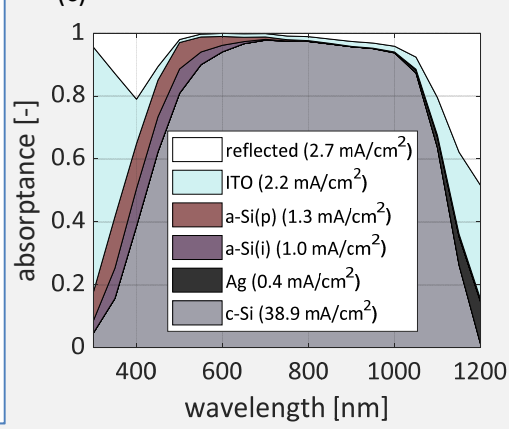


Fig. 3: a) Input file 'example3.m' for simulating a SHJ cell with pyramid texture. b) The pyramid texture used in the simulation. c) Simulation output: an area plot of R, T and A_i versus wavelength.

Example 4: Saving scatter matrices for fast parameter variation

In the previous example we set 'S.SM = 1' to store the scatter matrices in 'Int.SM'. In this example we do not clear 'Int'. This means that the scatter matrices are given as input and are reused instead of recalculated. As shown in Fig. 4a, a for-loop is used to repeat the simulation while varying the wafer thickness from 50 to 500 µm, in steps of 25 µm. Fig. 4b shows the implied photocurrent of the c-Si wafer as a function of its thickness. As expected, a thicker wafer absorbs more light and gives rise to a higher implied photocurrent. This is the result of 18 simulations (each of 19 wavelengths) and finished in about 10 seconds. This illustrates the speed gain that can be achieved by reusing the scatter matrices, a unique feature of GENPRO4.

Warning: When scatter matrices from a previous simulation exist in the input 'Int.SM', they will automatically be used. This means that any modifications to that interface (e.g. coating material or thickness, or texture morphology) are ignored until the scatter matrix is cleared. Clearing can be done for all interfaces simultaneously by the command 'clear Int'. It can also be done selectively, e.g. for interface 1 using 'Int(1).SM = [];' when only interface 1 is varied.

(a)

```
T = 50:25:500;                    %specify wafe
J = zeros(size(T));               %initialize

for t = 1:length(T)               %for every th
    disp(['thickness: ',num2str(T(t)),' um'])
    Lay(2).thi = T(t);            %adjust layer
    [Lay,Int] = GENPRO4(Lay,Int,S); %re-run s
    J(t) = Lay(2).cur;             %store curre
    drawnow
end

%plot current versus thickness
figure(1)
plot(T,J,'or')
xlabel('c-Si wafer thickness [\mum]')
ylabel('implied photocurrent [mA/cm^2]')
```

(b)

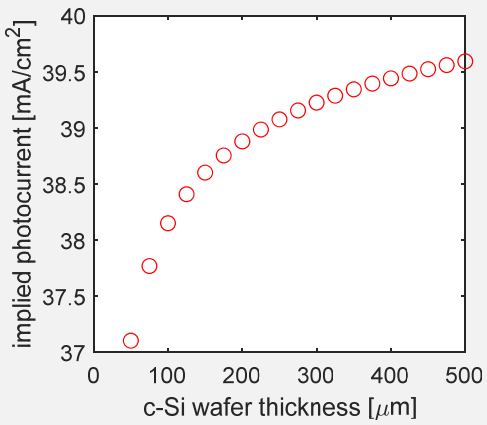


Fig. 4: a) Input file 'example4.m' for varying c-Si wafer thickness of the SHJ cell of example 3. b) Simulation output: implied photocurrent versus c-Si wafer thickness.

Example 5: Current matching perovskite / silicon tandem

The goal of this example is to simulate a perovskite / silicon tandem solar cell and find the perovskite thickness that gives identical implied photocurrents in perovskite and silicon. The input file is shown in Fig. 5a. The perovskite thin-film top cell is modelled as a set of coatings at the air/Si interface in order to take interference effects into account. The bottom cell is similar to the SHJ cell used in previous examples. It has pyramid texture at the rear, but is flat at the front side. The perovskite thickness is initially set to 400 nm, which gives 21.4 and 20.4 mA/cm^2 implied photocurrent in perovskite and silicon, respectively. The Matlab function 'fzero' is then used to find the perovskite thickness at which both currents are equal (their difference is zero). This turns out to be the case at 355 nm perovskite thickness and the result, shown in Fig. 5b, confirms that the implied photocurrent of perovskite and silicon are indeed identical at 20.8 mA/cm^2.

Note that 'fzero' calls the nested function 'match' (not shown) that runs GENPRO4 and calculates the mismatch current. To speed up the simulation the scatter matrix stored in 'Int(2).SM' is reused on every iteration (see example 4) because the variable 'Int' spans the child function 'match'. It is however essential that inside the function 'match' the scatter matrix of interface 1 is cleared 'Int(1).SM', because otherwise the variations in perovskite thickness would not be accounted for.

(a)

```
function [out,tm] = example5()
load('AFM.mat','pyramids_20um')  %load 20x20um height map

%===LAYERS===
Lay(1).med = 'air';                 Lay(1).thi = inf;
Lay(2).med = 'c-Si';                Lay(2).thi = 200;
Lay(3).med = 'air';                 Lay(3).thi = inf;

%===INTERFACES===
%interface 1: between layer 1 and 2 (air/c-Si)
Int(1).coat(1).med = 'MgF2';        Int(1).coat(1).thi = 0.100;
Int(1).coat(2).med = 'ITO';         Int(1).coat(2).thi = 0.040;
Int(1).coat(3).med = 'TiO2';        Int(1).coat(3).thi = 0.040;
Int(1).coat(4).med = 'perovskite';  Int(1).coat(4).thi = 0.400;
Int(1).coat(5).med = 'NiO';         Int(1).coat(5).thi = 0.040;
Int(1).coat(6).med = 'a-SiOx';      Int(1).coat(6).thi = 0.040;
Int(1).coat(7).med = 'a-Si(n)';     Int(1).coat(7).thi = 0.005;
Int(1).coat(8).med = 'a-Si(i)';     Int(1).coat(8).thi = 0.005;
%interface 2: between layer 2 and 3 (c-Si/air)
Int(2).model = 'ray';               %use RAY-optics model
Int(2).Z = -pyramids_20um;          Int(2).xy = [20,20];
Int(2).coat(1).med = 'a-Si(i)';     Int(2).coat(1).thi = 0.005;
Int(2).coat(2).med = 'a-Si(p)';     Int(2).coat(2).thi = 0.005;
Int(2).coat(3).med = 'Ag';          Int(2).coat(3).thi = 0.300;
%===

[tm] = fzero(@match,Int(1).coat(4).thi);  %find zero-mismatch
Int(1).coat(4).thi = tm;                  %set thickness
[Lay,Int,out] = GENPRO4(Lay,Int);         %run final result
```

(b)

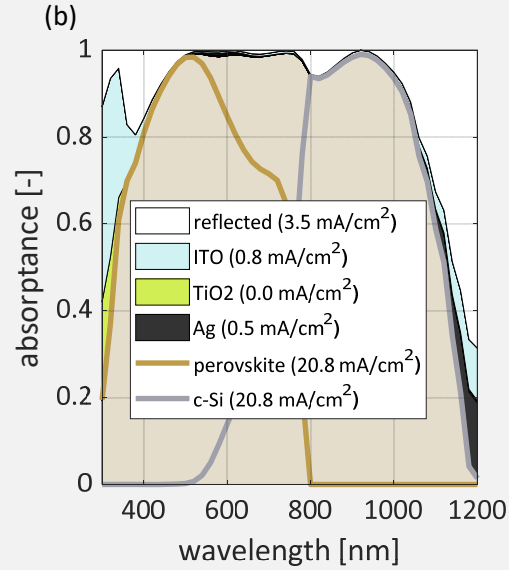


Fig.5: a) Input file 'example5.m' for current matching perovskite / silicon tandem b) Simulation output: an area plot of *R*, *T* and A_i versus wavelength.

The area plots are generated by the function 'aplot.m'. It displays *R*, *A* and *T* as a stacked area plot. Absorber layers like perovskite and silicon are always plotted at the bottom of the stack to allow comparison with EQE measurements. In case of multiple absorbers (as in tandems) the combined area and the individual lines are plotted as shown in Fig. 5b. To adjust the plotting style, 'aplot.m' can be edited.

3. Concluding remarks

The previous examples demonstrate the power and flexibility of GenPro4. The possibility of quickly calculating the implied photocurrent as a function of absorber layer thickness is especially useful for matching the current in monolithic tandem solar cells.

References

[1] R. Santbergen, T. Meguro, T. Suezaki, G. Koizumi, K. Yamamoto, M. Zeman, GenPro4 optical model for solar cell simulation and its application to multijunction solar cells, *IEEE Journal of Photovoltaics* **7** (2017) p. 919-926.

When using results from GenPro4 in a publication, please cite ref. 1.

Appendix A: Creating nk-files

The nk-folder provided with GENPRO4 holds many nk-files, each containing a table of *n* and *k* versus wavelength for one material. An nk-file is a plain text file with '.nk' extension and can be opened in a text editor such as Notepad. Only materials for which the nk-file is present can be used in the simulations. If another material is required, a corresponding nk-file has to be created and placed in the nk-folder. As shown in Fig. A1 below, the nk-file consists of four parts:

1. First column: here the wavelength is specified. The column heading can be 'nm', 'um' or 'm' depending whether the wavelength is specified in nm, µm or m. Alternatively it is possible to specify the photon energy in eV, in that case the column heading should be 'eV'.
2. Second column: here the refractive index *n* is specified, which is dimensionless. The column heading should be 'n'.
3. Third column: here the extinction coefficient *k* should be specified, which is dimensionless. The column heading should be 'k'. Alternatively it is possible to specify the absorption coefficient α. In that case the column heading should be 'alpha' and it is assumed that the units are cm^{-1}.
4. Heading (optional): here background information such as the source of the data can be provided. This information is not used by GENPRO4.

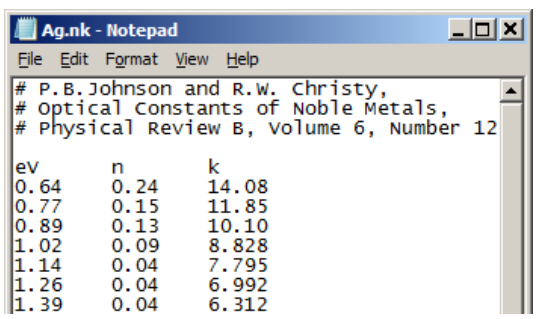


Fig. A1: Example of nk-file showing heading and data in three columns.

In case an error message appears when using a new nk-file for the first time, the file most likely contains redundant spaces or tabs that should be removed, especially near the top of the table. GENPRO4 uses 'readnk.m'. If needed, this file can be edited to make GENPRO4 accept other file formats.

In case a material is considered with a constant (not wavelength dependent) *n* and *k*, it is possible to specify the refractive index $N = n + ik$ directly in the input file. For example instead of writing 'ITO' one could use '1.85' or '1.85 + 0.03i' without requiring a corresponding nk-file. Note that the number should be given in quotation marks.

Note that after the simulation finishes, the refractive index data is stored in the 'Lay' and 'Int' structures. This allows the simulation to be repeated, for example inside a loop, without reading the nk-file over and over again, which saves time. If this behaviour is not desired, e.g. because the material is changed inside the loop, the stored data should be cleared before running the simulation.